

VQEs for H₂ Molecule Reproducing Results from PhysRevA.110.062414

Prashant Gupta (MSc)
Indian Institute of Technology Kharagpur

November 27, 2025

Contents

1	Abstract	4
2	Theoretical Background	4
2.1	The Electronic Structure Problem	4
2.2	Second Quantization and Jordan-Wigner Mapping	4
2.3	Variational Quantum Eigensolver (VQE)	5
2.4	Chemical Accuracy	5
3	Cell-by-Cell Analysis	5
3.1	Cell 1: Title and Introduction	5
3.2	Cell 2: Setup and Installation	6
3.3	Cell 3: Import Statements	6
3.3.1	System Configuration	6
3.3.2	Qiskit Core	7
3.3.3	Primitives and Algorithms	7
3.3.4	Quantum Chemistry	7
3.3.5	Mathematical Tools	7
3.4	Cell 4: Helper Function - <code>get_h2_problem</code>	8
3.4.1	Molecule Specification	8
3.4.2	Basis Set Choice	9
3.4.3	Molecular Properties	9
3.4.4	Problem Generation	9
3.4.5	Jordan-Wigner Transformation	9
3.5	Cell 5: Building the XY Hamiltonian	10
3.5.1	XY Model	11
3.5.2	Power-Law Coupling	11
3.5.3	Pauli String Construction	12
3.6	Cell 6: Building HEA-TI Ansatz	12
3.6.1	Hartree-Fock Initial State	13
3.6.2	Global Entangling Gate Construction	13
3.6.3	Evolution Time Parameter	13

3.6.4	Layer Construction	14
3.7	Cell 7: Problem Setup	14
3.7.1	Bond Distance Choice	15
3.7.2	Number of Layers	15
3.7.3	Exact Diagonalization	15
3.7.4	Nuclear Repulsion Energy	16
3.8	Cell 8: VQE with UCCSD Ansatz	16
3.8.1	UCCSD Theory	17
3.8.2	VQE Setup	18
3.8.3	Convergence Tracking	18
3.8.4	Initial Parameters	19
3.8.5	Running VQE	19
3.9	Cell 9: VQE with TwoLocal Ansatz	20
3.9.1	TwoLocal Ansatz Theory	20
3.9.2	Ansatz Composition	22
3.9.3	Initialization Strategy	22
3.9.4	Expected Performance	22
3.10	Cell 10: VQE with HEA-TI Ansatz	23
3.10.1	Key Differences from Previous Ansätze	23
3.10.2	Parameter Count Comparison	23
3.10.3	Convergence Behavior	24
3.10.4	Physical Interpretation of Results	24
3.11	Cell 11: Results Summary and Comparison	24
3.11.1	Error Analysis	25
3.11.2	Comparison of Results	25
3.11.3	Why All Methods Succeed for H_2	26
3.12	Cell 12: Convergence Visualization	26
3.12.1	Interpretation of Convergence Curves	26
3.13	Cell 13: Error Bar Charts	27
4	Connection to Paper Results	28
4.1	Paper Figure 3: Error vs. Circuit Depth	28
4.2	Paper Figure 4(a): Potential Energy Curve	28
4.3	Paper Section IV: Quantum Chemistry Results	29
4.4	Paper Section V: Error Analysis	29
5	Implementation Details	30
5.1	Fixed Compatibility Issues	30
5.1.1	Issue 1: Deprecated Estimator	30
5.1.2	Issue 2: QubitConverter Deprecation	30
5.1.3	Issue 3: TwoLocal Import	30
5.2	Optimization Considerations	31
5.2.1	Optimizer Choice	31
5.2.2	Initialization Strategies	31
5.3	Numerical Stability	32
5.3.1	Matrix Exponentiation	32
5.3.2	Energy Measurement	32

6	Extensions and Future Work	32
6.1	Extending to More Molecules	32
6.2	Adding Noise Models	33
6.3	Error Mitigation	33
6.4	Advanced Optimizers	33
7	Troubleshooting	34
7.1	Common Errors and Solutions	34
7.1.1	Import Errors	34
7.1.2	Version Incompatibility	34
7.2	Performance Optimization	34
7.2.1	Speed Up Simulations	34
8	Conclusion	34
8.1	Key Takeaways	34
8.2	Paper Contributions Validated	35
8.3	Final Remarks	35
A	Glossary	36
B	Mathematical Notation	36

1 Abstract

This report provides a comprehensive, cell-by-cell analysis of a Jupyter notebook implementing the Variational Quantum Eigensolver (VQE) algorithm for calculating the ground state energy of the hydrogen molecule (H_2). The implementation reproduces results from the paper “*Hardware-efficient variational quantum algorithm in a trapped-ion quantum computer*” (Physical Review A 110, 062414, 2024) by Zhuang et al.

The notebook implements three different variational ansätze:

1. **UCCSD** (Unitary Coupled Cluster Singles and Doubles) - The gold standard in quantum chemistry
2. **TwoLocal** - A standard hardware-efficient ansatz
3. **HEA-TI** (Hardware-Efficient Ansatz for Trapped Ions) - The paper’s custom ansatz

Each ansatz is evaluated for its ability to achieve *chemical accuracy* (error < 0.0015 Hartree = 1.5 millihartree) for the H_2 molecule at equilibrium bond distance (0.74 Å).

2 Theoretical Background

2.1 The Electronic Structure Problem

The electronic structure problem seeks to find the ground state energy of a molecular system described by the electronic Hamiltonian:

$$\hat{H} = - \sum_i \frac{\nabla_i^2}{2} - \sum_{i,A} \frac{Z_A}{|\mathbf{r}_i - \mathbf{R}_A|} + \sum_{i<j} \frac{1}{|\mathbf{r}_i - \mathbf{r}_j|} + \sum_{A<B} \frac{Z_A Z_B}{|\mathbf{R}_A - \mathbf{R}_B|} \quad (1)$$

where the terms represent:

- Kinetic energy of electrons
- Electron-nuclear attraction
- Electron-electron repulsion
- Nuclear-nuclear repulsion (constant for fixed geometry)

For the H_2 molecule, this simplifies significantly as we have only 2 electrons and 2 protons.

2.2 Second Quantization and Jordan-Wigner Mapping

The molecular Hamiltonian in second quantization becomes:

$$\hat{H} = \sum_{i,j} h_{ij} \hat{a}_i^\dagger \hat{a}_j + \frac{1}{2} \sum_{i,j,k,l} g_{ijkl} \hat{a}_i^\dagger \hat{a}_j^\dagger \hat{a}_k \hat{a}_l \quad (2)$$

The Jordan-Wigner transformation maps fermionic operators to qubit (Pauli) operators:

$$\hat{a}_j \rightarrow \frac{1}{2}(X_j + iY_j) \prod_{k=1}^{j-1} Z_k \quad (3)$$

This allows us to represent the molecular Hamiltonian as a sum of Pauli strings that can be measured on a quantum computer.

2.3 Variational Quantum Eigensolver (VQE)

VQE is a hybrid quantum-classical algorithm that exploits the variational principle:

$$E_0 \leq \frac{\langle \psi(\theta) | \hat{H} | \psi(\theta) \rangle}{\langle \psi(\theta) | \psi(\theta) \rangle} \quad (4)$$

The algorithm iteratively:

1. Prepares a parameterized quantum state $|\psi(\theta)\rangle$ using a quantum circuit (ansatz)
2. Measures the expectation value $\langle \hat{H} \rangle$
3. Uses a classical optimizer to update parameters θ to minimize energy

2.4 Chemical Accuracy

Chemical accuracy is defined as an error of less than 0.0015 Hartree (1.5 millihartree), which corresponds to approximately:

- 0.043 eV
- 1 kcal/mol
- 350 cm^{-1}

This level of accuracy is sufficient for most practical chemistry applications.

3 Cell-by-Cell Analysis

3.1 Cell 1: Title and Introduction

Purpose: Provides overview and context for the notebook.

Content: This cell contains a markdown header explaining that the notebook implements VQE for H_2 using three different ansätze to reproduce results from Phys-RevA.110.062414.

Key Points:

- UCCSD serves as the benchmark (gold standard)
- TwoLocal represents standard hardware-efficient approaches
- HEA-TI is the paper’s novel contribution
- All compatibility issues with latest Qiskit versions have been fixed

3.2 Cell 2: Setup and Installation

Purpose: Install all required Python packages for quantum computing and quantum chemistry simulations.

Code:

```
1 !pip install qiskit qiskit-nature qiskit-nature-pyscf qiskit-  
    algorithms pyscf matplotlib numpy scipy -q
```

Packages Explained:

- **qiskit:** Core quantum computing framework by IBM
- **qiskit-nature:** Quantum chemistry applications module
- **qiskit-nature-pyscf:** Integration with PySCF classical chemistry package
- **qiskit-algorithms:** VQE and optimization algorithms
- **pyscf:** Classical quantum chemistry library for generating molecular Hamiltonians
- **matplotlib:** Plotting and visualization
- **numpy:** Numerical computations
- **scipy:** Scientific computing (used for matrix exponentiation)

Why These Packages:

- PySCF computes molecular integrals h_{ij} and g_{ijkl} classically
- Qiskit Nature handles Jordan-Wigner transformation
- Qiskit provides quantum circuit simulation capabilities

3.3 Cell 3: Import Statements

Purpose: Import all necessary modules and configure environment.

Key Imports Explained:

3.3.1 System Configuration

```
1 import os  
2 os.environ['KMP_WARNINGS'] = '0'  
3 import warnings  
4 warnings.filterwarnings('ignore')
```

Explanation: Suppresses PySCF and NumPy warnings that clutter output without providing useful information.

3.3.2 Qiskit Core

```
1 from qiskit import QuantumCircuit
2 from qiskit.circuit import Parameter
3 from qiskit.circuit.library import UnitaryGate
```

Explanation:

- `QuantumCircuit`: Base class for building quantum circuits
- `Parameter`: Creates parameterized gates for variational optimization
- `UnitaryGate`: Allows custom unitary matrices as gates (used for HEA-TI)

3.3.3 Primitives and Algorithms

```
1 from qiskit.primitives import StatevectorEstimator
2 from qiskit_algorithms import VQE, NumPyMinimumEigensolver
3 from qiskit_algorithms.optimizers import SLSQP
```

Explanation:

- `StatevectorEstimator`: Computes expectation values using statevector simulation (exact, no sampling noise)
- `VQE`: Variational Quantum Eigensolver implementation
- `NumPyMinimumEigensolver`: Classical exact diagonalization for benchmarking
- `SLSQP`: Sequential Least Squares Programming optimizer (gradient-based)

3.3.4 Quantum Chemistry

```
1 from qiskit_nature.second_q.drivers import PySCFDriver
2 from qiskit_nature.second_q.mappers import JordanWignerMapper
3 from qiskit_nature.second_q.circuit.library import UCCSD,
   HartreeFock
```

Explanation:

- `PySCFDriver`: Interface to PySCF for molecular calculations
- `JordanWignerMapper`: Implements Jordan-Wigner transformation
- `UCCSD`: Pre-built UCCSD ansatz circuit
- `HartreeFock`: Initial state preparation (mean-field approximation)

3.3.5 Mathematical Tools

```
1 from scipy.linalg import expm
2 import numpy as np
```

Explanation:

- `expm`: Matrix exponential e^{-iHt} (crucial for HEA-TI)
- `numpy`: Array operations and random number generation

3.4 Cell 4: Helper Function - get_h2_problem

Purpose: Define the H₂ molecule and generate its qubit Hamiltonian.

Complete Code:

```
1 def get_h2_problem(bond_distance: float):
2     """
3     Defines the H2 molecule problem in Qiskit Nature.
4
5     Args:
6         bond_distance: H-H bond distance in Angstroms
7
8     Returns:
9         qubit_op: Qubit Hamiltonian
10        problem: Electronic structure problem
11        num_qubits: Number of qubits
12    """
13    print(f'Setting up H2 molecule problem at d={bond_distance}
14          Angstrom')
15
16    # Use PySCF to define the molecule
17    driver = PySCFDriver(
18        atom=f'H 0 0 0; H 0 0 {bond_distance}',
19        basis='sto3g',
20        charge=0,
21        spin=0,
22        unit=DistanceUnit.ANGSTROM
23    )
24
25    # Get the electronic structure problem
26    problem = driver.run()
27
28    # Map to qubits using Jordan-Wigner
29    mapper = JordanWignerMapper()
30    second_q_op = problem.hamiltonian.second_q_op()
31    qubit_op = mapper.map(second_q_op)
32
33    num_qubits = qubit_op.num_qubits
34    print(f'Problem mapped to {num_qubits} qubits')
35
36    return qubit_op, problem, num_qubits, mapper
```

Detailed Explanation:

3.4.1 Molecule Specification

```
1 atom=f'H 0 0 0; H 0 0 {bond_distance}'
```

Meaning: Defines two hydrogen atoms:

- First H at origin: (0,0,0)
- Second H at: (0,0,*d*) where *d* is the bond distance

This places atoms along the z-axis with separation d Å.

3.4.2 Basis Set Choice

```
1 basis='sto3g'
```

Explanation: STO-3G (Slater Type Orbital with 3 Gaussians) is the minimal basis set:

- Each hydrogen atom has 1 atomic orbital (1s)
- Each orbital is approximated by 3 Gaussian functions
- For H₂: 2 atoms $\times$ 1 orbital = 2 spatial orbitals
- With spin: 2 spatial $\times$ 2 spin states = 4 spin orbitals
- Maps to 4 qubits after Jordan-Wigner transformation

3.4.3 Molecular Properties

```
1 charge=0, spin=0
```

Meaning:

- charge=0: Neutral molecule (2 protons, 2 electrons)
- spin=0: Singlet state (total spin $S = 0$, closed shell)

3.4.4 Problem Generation

```
1 problem = driver.run()  
2 second_q_op = problem.hamiltonian.second_q_op()
```

What Happens:

1. PySCF performs Hartree-Fock calculation
2. Computes one-electron integrals h_{ij} and two-electron integrals g_{ijkl}
3. Constructs second-quantized Hamiltonian:

$$\hat{H} = \sum_{ij} h_{ij} a_i^\dagger a_j + \frac{1}{2} \sum_{ijkl} g_{ijkl} a_i^\dagger a_j^\dagger a_k a_l \quad (5)$$

3.4.5 Jordan-Wigner Transformation

```
1 mapper = JordanWignerMapper()  
2 qubit_op = mapper.map(second_q_op)
```

Transformation Rules:

$$a_j \rightarrow \frac{1}{2}(X_j + iY_j) \prod_{k < j} Z_k \quad (6)$$

$$a_j^\dagger \rightarrow \frac{1}{2}(X_j - iY_j) \prod_{k < j} Z_k \quad (7)$$

Result: Qubit Hamiltonian as sum of Pauli strings:

$$H_{qubit} = \sum_i c_i P_i \quad (8)$$

where $P_i \in \{I, X, Y, Z\}^{\otimes 4}$ and $c_i \in \mathbb{R}$.

For H_2 , this typically involves ~ 15 -20 Pauli terms.

3.5 Cell 5: Building the XY Hamiltonian

Purpose: Construct the global entangling Hamiltonian for the HEA-TI ansatz.

Code:

```

1 def build_xy_hamiltonian(num_qubits: int, J0: float = 1.0, alpha:
2   float = 1.5):
3   """
4   Builds the H_XY Hamiltonian for the global entangling gate.
5
6   H_XY = sum_{i<j} J_ij (X_i X_j + Y_i Y_j)
7   where J_ij = J_0 / |i-j|^alpha
8   """
9   pauli_list = []
10
11   for i in range(num_qubits):
12       for j in range(i + 1, num_qubits):
13           # Power-law coupling
14           dist = abs(i - j)
15           J_ij = J0 / (dist ** alpha)
16
17           # XX term
18           xx_pauli = ['I'] * num_qubits
19           xx_pauli[i] = 'X'
20           xx_pauli[j] = 'X'
21           pauli_list.append(''.join(xx_pauli), J_ij))
22
23           # YY term
24           yy_pauli = ['I'] * num_qubits
25           yy_pauli[i] = 'Y'
26           yy_pauli[j] = 'Y'
27           pauli_list.append(''.join(yy_pauli), J_ij))
28
29   return SparsePauliOp.from_list(pauli_list)

```

Detailed Theory:

3.5.1 XY Model

The XY Hamiltonian is defined as:

$$H_{XY} = \sum_{i < j} J_{ij} (\sigma_i^+ \sigma_j^- + \sigma_i^- \sigma_j^+) \quad (9)$$

where $\sigma^+ = \frac{1}{2}(X + iY)$ and $\sigma^- = \frac{1}{2}(X - iY)$ are raising and lowering operators. This can be rewritten in terms of Pauli matrices:

$$H_{XY} = \sum_{i < j} J_{ij} (X_i X_j + Y_i Y_j) \quad (10)$$

Why XY Model: This Hamiltonian:

- Preserves total magnetization: $[H_{XY}, \sum_i Z_i] = 0$
- Creates entanglement between all qubit pairs
- Naturally available in trapped-ion systems via phonon-mediated interactions

3.5.2 Power-Law Coupling

```
1 J_ij = J0 / (dist ** alpha)
```

The coupling strength decays as:

$$J_{ij} = \frac{J_0}{|i - j|^\alpha} \quad (11)$$

Parameters from Paper:

- $J_0 = 1$ (overall coupling strength)
- $\alpha = 1.5$ (power-law exponent, typical for trapped ions)

Example for 4 qubits:

$$\begin{aligned} J_{01} &= J_0 / 1^{1.5} = 1.000 \\ J_{02} &= J_0 / 2^{1.5} = 0.354 \\ J_{03} &= J_0 / 3^{1.5} = 0.192 \\ J_{12} &= J_0 / 1^{1.5} = 1.000 \\ J_{13} &= J_0 / 2^{1.5} = 0.354 \\ J_{23} &= J_0 / 1^{1.5} = 1.000 \end{aligned}$$

Physical Meaning: Nearest neighbors interact most strongly, with long-range interactions decaying smoothly.

3.5.3 Pauli String Construction

```

1 xx_pauli = ['I'] * num_qubits
2 xx_pauli[i] = 'X'
3 xx_pauli[j] = 'X'
4 pauli_list.append((''.join(xx_pauli), J_ij))

```

Example: For qubits $i = 1, j = 3$ with 4 qubits total:

- XX term: 'IXIX' $\equiv I \otimes X \otimes I \otimes X$
- YY term: 'IYIY' $\equiv I \otimes Y \otimes I \otimes Y$

3.6 Cell 6: Building HEA-TI Ansatz

Purpose: Construct the complete HEA-TI circuit as described in the paper.

Circuit Structure (from Paper Section II.B):

The HEA-TI ansatz has D layers, each consisting of:

1. Single-qubit RZ rotations: $\prod_{i=1}^N R_z^{(i)}(\theta_d^i)$
2. Global entangling gate: $U_G(t_d) = e^{-iH_{XY}t_d}$

Mathematical Form:

$$U(\theta) = \prod_{d=1}^D \left[U_G(t_d) \prod_{i=1}^N R_z^{(i)}(\theta_d^i) \right] \quad (12)$$

Applied to Hartree-Fock initial state:

$$|\psi(\theta)\rangle = U(\theta)|HF\rangle \quad (13)$$

Code Analysis:

```

1 def build_he_a_ti_ansatz(num_qubits, num_particles, mapper,
2                           num_layers=4, t_d=0.4):
3     # Initialize with Hartree-Fock state
4     init_state = HartreeFock(
5         num_spatial_orbitals=num_qubits // 2,
6         num_particles=num_particles,
7         qubit_mapper=mapper
8     )
9
10    qc = QuantumCircuit(num_qubits)
11    qc.compose(init_state, inplace=True)
12    qc.barrier()

```

3.6.1 Hartree-Fock Initial State

For H_2 :

- 2 electrons, 4 spin orbitals (qubits)
- Electron configuration: $|1100\rangle$ in computational basis
- First two orbitals (spin-up and spin-down of lowest energy molecular orbital) are occupied

Why Start from HF:

- Mean-field approximation captures $\sim 90\text{-}99\%$ of correlation energy
- Provides good initial guess, reducing optimization difficulty
- Satisfies correct particle number and spin symmetry

3.6.2 Global Entangling Gate Construction

```
1 H_xy = build_xy_hamiltonian(num_qubits)
2 H_xy_matrix = H_xy.to_matrix()
3 U_G_matrix = expm(-1j * t_d * H_xy_matrix)
4 U_G_gate = UnitaryGate(U_G_matrix, label=f'U_G(t={t_d})')
```

Step-by-Step:

1. **Build Hamiltonian:** Construct H_{XY} as SparsePauliOp
2. **Convert to Matrix:** $H_{XY} \rightarrow 16 \times 16$ complex matrix (for 4 qubits)
3. **Matrix Exponential:** Compute $U_G = e^{-iH_{XY}t_d}$ using scipy
4. **Create Gate:** Wrap matrix as UnitaryGate for circuit use

Computational Note:

- Matrix exponential is computed *once* per ansatz
- Complexity: $O(2^{3n})$ for n qubits
- For H_2 (4 qubits): $16^3 = 4096$ operations - very fast
- Same gate reused across all layers (no recomputation needed)

3.6.3 Evolution Time Parameter

```
1 t_d = 0.4 # Fixed from paper Section IV
```

From Paper:

“For simplicity and experimental convenience, we fix the evolution time of all layers to a constant $t_d \equiv t_0 = 0.4$ ”

Physical Units: In trapped-ion experiments with $J_0 = 2\pi \times 0.5$ kHz:

$$t_{\text{physical}} = \frac{t_d}{2\pi J_0} = \frac{0.4}{2\pi \times 500 \text{ Hz}} \approx 0.13 \text{ ms} \quad (14)$$

3.6.4 Layer Construction

```
1 for d in range(num_layers):
2     # Parameterized RZ rotations
3     for i in range(num_qubits):
4         param = Parameter(f'theta_{d}_{i}')
5         qc.rz(param, i)
6         parameters.append(param)
7
8     qc.barrier()
9
10    # Add global entangling gate
11    qc.append(U_G_gate, range(num_qubits))
12    qc.barrier()
```

Parameter Count:

- Each layer has N RZ rotation angles (one per qubit)
- D layers $\Rightarrow D \times N$ total parameters
- For H_2 with $D = 4$: $4 \times 4 = 16$ parameters

Circuit Depth:

- Each layer: 1 (parallel RZ) + 1 (global gate) = 2 depth units
- D layers $\Rightarrow$ depth $\approx 2D$
- For $D = 4$: depth ≈ 8 (plus HF initialization)

Why This Works (from Paper):

1. **Expressiveness:** RZ + XY entanglement can approximate any state
2. **Hardware Efficiency:** Single global operation (no individual two-qubit gates)
3. **Charge Conservation:** RZ preserves $\sum_i Z_i$, suitable for chemistry

3.7 Cell 7: Problem Setup

Purpose: Define the specific H_2 molecule instance and calculate exact ground state for comparison.

Code:

```
1 bond_distance = 0.74    # Equilibrium bond distance for H2
2 hea_ti_layers = 4       # D=4 from paper Figure 3
3
4 # Define problem
5 qubit_op, problem, num_qubits, mapper = get_h2_problem(
6     bond_distance)
7
8 # Calculate exact ground state (benchmark)
9 np_solver = NumPyMinimumEigensolver()
```

```

9 exact_result = np_solver.compute_minimum_eigenvalue(qubit_op)
10 exact_energy = exact_result.eigenvalue
11
12 # Add nuclear repulsion
13 nuc_rep_energy = problem.hamiltonian.nuclear_repulsion_energy
14 total_exact_energy = exact_energy + nuc_rep_energy

```

3.7.1 Bond Distance Choice

```

1 bond_distance = 0.74

```

Physical Meaning:

- Equilibrium bond distance for H₂ molecule
- At this distance, molecule has minimum energy (most stable)
- Experimental value: ≈ 0.74 Å

Potential Energy Surface:

$d < 0.74$ Å : atoms too close, repulsion dominates

$d = 0.74$ Å : equilibrium, minimum energy

$d > 0.74$ Å : atoms separating, dissociation

3.7.2 Number of Layers

```

1 hea_ti_layers = 4

```

From Paper Figure 3:

“For H₂ (4 qubits), chemical accuracy is achieved with $D = 4$ layers”

Why $D = 4$ is Sufficient:

- H₂ has weak electron correlation (mostly single-reference)
- 16 variational parameters provide enough flexibility
- Higher D gives diminishing returns (see paper Figure 3)

3.7.3 Exact Diagonalization

```

1 np_solver = NumPyMinimumEigensolver()
2 exact_result = np_solver.compute_minimum_eigenvalue(qubit_op)

```

What This Does:

1. Converts qubit Hamiltonian to 16×16 matrix (4 qubits $\rightarrow 2^4$ states)
2. Performs classical diagonalization: finds eigenvalues $\{\lambda_i\}$
3. Returns minimum eigenvalue: $E_0 = \min_i \lambda_i$

Computational Complexity:

- Matrix diagonalization: $O(2^{3n})$ for n qubits
- For H_2 : $16^3 = 4096$ operations - instant
- For LiH (6 qubits): $64^3 = 262,144$ - still fast
- For F_2 (12 qubits): 4096^3 - challenging but feasible

Why We Can Do This: Classical exact diagonalization is only possible for small systems. For larger molecules (> 20 qubits), this becomes impossible, highlighting the need for VQE.

3.7.4 Nuclear Repulsion Energy

```
1 nuc_rep_energy = problem.hamiltonian.nuclear_repulsion_energy
2 total_exact_energy = exact_energy + nuc_rep_energy
```

Physical Meaning:

$$E_{nuclear} = \frac{Z_A Z_B}{|\mathbf{R}_A - \mathbf{R}_B|} \quad (15)$$

For H_2 : Both nuclei have charge $Z = 1$:

$$E_{nuclear} = \frac{1 \times 1}{d} = \frac{1}{0.74 \text{ \AA}} \approx 1.351 \text{ Hartree} \quad (16)$$

Why Separate:

- Electronic Hamiltonian only includes electron-electron and electron-nuclear terms
- Nuclear repulsion is constant for fixed geometry (Born-Oppenheimer approximation)
- Total energy = Electronic energy + Nuclear repulsion

Typical Values:

- Electronic energy: ~ -1.85238817 Hartree (negative, attractive)
- Nuclear repulsion: $\sim +0.71510434$ Hartree (positive, repulsive)
- Total energy: ~ -1.13728383 Hartree (bound state)

3.8 Cell 8: VQE with UCCSD Ansatz

Purpose: Run VQE using the gold-standard UCCSD ansatz for benchmarking.

3.8.1 UCCSD Theory

The Unitary Coupled Cluster Singles and Doubles (UCCSD) ansatz is based on quantum chemistry's coupled cluster method:

$$|\psi_{UCCSD}\rangle = e^{\hat{T}-\hat{T}^\dagger}|HF\rangle \quad (17)$$

where the cluster operator includes singles and doubles excitations:

$$\hat{T} = \hat{T}_1 + \hat{T}_2 = \sum_{i,a} t_i^a \hat{a}_a^\dagger \hat{a}_i + \sum_{i<j, a<b} t_{ij}^{ab} \hat{a}_a^\dagger \hat{a}_b^\dagger \hat{a}_j \hat{a}_i \quad (18)$$

Components:

- $\hat{T}_1$: Single excitations (electron from occupied i to virtual a)
- $\hat{T}_2$: Double excitations (two electrons excited simultaneously)
- t_i^a, t_{ij}^{ab} : Variational amplitudes (optimized parameters)

For H_2 :

- 2 occupied orbitals (0,1), 2 virtual orbitals (2,3)
- Single excitations: $0 \rightarrow 2, 0 \rightarrow 3, 1 \rightarrow 2, 1 \rightarrow 3$ (4 terms)
- Double excitations: $(0,1) \rightarrow (2,3)$ (1 term)
- Total: 5 cluster operators $\Rightarrow$ typically 3 independent parameters after symmetry

Code Analysis:

```

1 init_state = HartreeFock(
2     num_spatial_orbitals=problem.num_spatial_orbitals,
3     num_particles=problem.num_particles,
4     qubit_mapper=mapper
5 )
6
7 uccsd_ansatz = UCCSD(
8     num_spatial_orbitals=problem.num_spatial_orbitals,
9     num_particles=problem.num_particles,
10    qubit_mapper=mapper,
11    initial_state=init_state
12 )

```

What UCCSD() Does:

1. Identifies all possible single and double excitations
2. Constructs excitation operators: $\hat{T}_1^i, \hat{T}_2^{ij}$
3. Maps to Pauli operators via Jordan-Wigner
4. Builds Trotterized circuit: $e^{\hat{T}-\hat{T}^\dagger} \approx \prod_k e^{i\theta_k(\hat{T}_k-\hat{T}_k^\dagger)}$

Circuit Properties:

- Parameters: 3 for H_2 (fewer than HEA-TT's 16)
- Depth: Relatively deep due to Trotterization
- Gates: Requires many two-qubit CNOT gates

3.8.2 VQE Setup

```
1 estimator = StatevectorEstimator()  
2 optimizer = SLSQP(maxiter=500)
```

StatevectorEstimator:

- Computes expectation values exactly (no sampling noise)
- For state $|\psi\rangle$ and observable $\hat{O}$: calculates $\langle\psi|\hat{O}|\psi\rangle$
- Decomposes $\hat{O} = \sum_i c_i P_i$ where P_i are Pauli strings
- Evaluates each $\langle\psi|P_i|\psi\rangle$ and sums: $\langle\hat{O}\rangle = \sum_i c_i \langle P_i \rangle$

SLSQP Optimizer:

- Sequential Least Squares Programming
- Gradient-based optimization (uses derivatives)
- Handles constrained optimization problems
- Converges faster than gradient-free methods (COBYLA, SPSA)

Why SLSQP:

- VQE gradients available via parameter-shift rule
- More efficient than gradient-free for small parameter counts
- Well-suited for smooth optimization landscapes

3.8.3 Convergence Tracking

```
1 energy_history_uccsd = []  
2 def callback_uccsd(eval_count, params, mean, std):  
3     energy_history_uccsd.append(mean + nuc_rep_energy)
```

Callback Function:

- Called after each optimizer iteration
- `eval_count`: Number of function evaluations so far
- `params`: Current parameter values θ_i
- `mean`: Current energy expectation $\langle\hat{H}\rangle$
- `std`: Standard deviation (0 for statevector simulation)

Purpose: Track energy vs. iteration to visualize convergence behavior.

3.8.4 Initial Parameters

```
1 initial_point=np.zeros(uccsd_ansatz.num_parameters)
```

Zero Initialization:

- For UCCSD: $\theta_i = 0 \Rightarrow e^0 = I \Rightarrow |\psi\rangle = |HF\rangle$
- Starts from Hartree-Fock solution
- Good initial guess (already $\sim 90\%$ accurate)
- Optimization refines to exact ground state

3.8.5 Running VQE

```
1 vqe_uccsd = VQE(  
2     estimator=estimator,  
3     ansatz=uccsd_ansatz,  
4     optimizer=optimizer,  
5     initial_point=initial_point,  
6     callback=callback_uccsd  
7 )  
8  
9 vqe_uccsd_result = vqe_uccsd.compute_minimum_eigenvalue(qubit_op)
```

VQE Algorithm Flow:

1. **Initialize:** Set $\theta = \theta_0$ (zeros for UCCSD)
2. **Prepare State:** Build circuit $|\psi(\theta)\rangle$
3. **Measure Energy:** Compute $E(\theta) = \langle\psi(\theta)|\hat{H}|\psi(\theta)\rangle$
4. **Compute Gradient:** Calculate $\nabla_\theta E(\theta)$ via parameter-shift
5. **Update:** $\theta \leftarrow \theta - \eta \nabla_\theta E(\theta)$ (learning rate η)
6. **Repeat:** Steps 2-5 until convergence or max iterations

Parameter-Shift Rule: For gate $U(\theta) = e^{-i\theta G}$:

$$\frac{\partial}{\partial \theta} \langle \hat{O} \rangle = \frac{1}{2} \left[\langle \hat{O} \rangle_{\theta+\pi/2} - \langle \hat{O} \rangle_{\theta-\pi/2} \right] \quad (19)$$

Result Interpretation:

```
1 vqe_uccsd_energy = vqe_uccsd_result.eigenvalue  
2 total_uccsd_energy = vqe_uccsd_energy + nuc_rep_energy
```

Expected Results for H_2 :

- Electronic energy: ≈ -1.85238817 Hartree
- Nuclear repulsion: $\approx +0.71510434$ Hartree
- Total energy: ≈ -1.13728383 Hartree
- Error from exact: 0.00000000 Hartree (excellent)
- Evaluations: ~ 10 (few parameters, good initialization)

3.9 Cell 9: VQE with TwoLocal Ansatz

Purpose: Implement standard hardware-efficient ansatz for comparison.

3.9.1 TwoLocal Ansatz Theory

The TwoLocal ansatz is a general hardware-efficient circuit structure:

$$U_{TwoLocal} = \prod_{d=1}^D \left[U_{ent} \prod_{i=1}^N R(\theta_{d,i}) \right] \quad (20)$$

Components:

- $R(\theta)$: Single-qubit rotation layer (RY, RZ, or both)
- U_{ent} : Entanglement layer (CNOT gates in some pattern)
- D : Number of repetitions (circuit depth)

Code Analysis:

```
1 twolocal_var = TwoLocalCircuit(  
2     num_qubits=num_qubits,  
3     rotation_blocks=['ry', 'rz'],  
4     entanglement_blocks='cx',  
5     entanglement='linear',  
6     reps=3  
7 )
```

Parameter Breakdown:

Rotation Blocks

```
1 rotation_blocks=['ry', 'rz']
```

Meaning: Each rotation layer applies:

$$R_i(\theta, \phi) = R_y^{(i)}(\theta_i) R_z^{(i)}(\phi_i) \quad (21)$$

Why RY and RZ:

- Together can generate any SU(2) rotation on single qubit
- R_y creates superpositions: $|0\rangle \leftrightarrow |1\rangle$
- R_z adjusts phases: $|\pm\rangle \rightarrow e^{\pm i\phi} |\pm\rangle$
- More expressive than RZ alone

Entanglement Blocks

```
1 entanglement_blocks='cx'
```

CNOT Gate:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad (22)$$

Action: $|a, b\rangle \rightarrow |a, a \oplus b\rangle$

Why CNOT:

- Universal two-qubit gate (with single-qubit gates)
- Creates entanglement: $|0+\rangle \rightarrow |00\rangle + |11\rangle$ (Bell state)
- Native gate in many quantum platforms

Entanglement Strategy

```
1 entanglement='linear'
```

Linear Entanglement: CNOTs applied in chain:

$$\text{CNOT}_{0,1}, \text{CNOT}_{1,2}, \text{CNOT}_{2,3} \quad (23)$$

Alternative Patterns:

- 'full': All-to-all CNOTs $\binom{N}{2}$ gates
- 'circular': Linear + wraparound ($0 \rightarrow N - 1$)
- 'sca': Strongly connected architecture

Linear Choice: Balances expressiveness with gate count.

Repetitions

```
1 reps=3
```

Circuit Structure:

Rotation Layer 1
CNOT Layer
Rotation Layer 2
CNOT Layer
Rotation Layer 3
CNOT Layer
Final Rotation Layer

Parameter Count:

- Each rotation layer: $2N$ parameters (RY + RZ on each qubit)
- Repetitions: $\text{reps} + 1 = 4$ rotation layers
- Total: $4 \times 2 \times 4 = 32$ parameters

3.9.2 Ansatz Composition

```
1 twolocal_ansatz = QuantumCircuit(num_qubits)
2 twolocal_ansatz.compose(init_state, inplace=True)
3 twolocal_ansatz.barrier()
4 twolocal_ansatz.compose(twolocal_var, inplace=True)
```

Full Circuit:

$$|\psi\rangle = U_{TwoLocal}|HF\rangle \quad (24)$$

Why Compose:

- `init_state`: Prepares Hartree-Fock (electronic configuration)
- `twolocal_var`: Variational circuit (optimization happens here)
- `barrier`: Visual separator (no effect on computation)

3.9.3 Initialization Strategy

```
1 initial_point=np.random.randn(twolocal_ansatz.num_parameters) *
  0.1
```

Random Small Initialization:

- $\theta_i \sim \mathcal{N}(0, 0.1^2)$ (Gaussian distribution)
- Small values prevent large deviations from HF state initially
- Provides symmetry breaking (avoids saddle points)

Contrast with UCCSD:

- UCCSD: Zero init = HF state (exact theory known)
- TwoLocal: No theoretical guidance $\rightarrow$ random initialization

3.9.4 Expected Performance

TwoLocal vs. UCCSD:

- **Parameters:** TwoLocal (32) > UCCSD (3)
- **Accuracy:** Comparable if sufficient depth
- **Hardware:** TwoLocal more native to superconducting qubits
- **Optimization:** TwoLocal may need more iterations (more parameters)

Expected Results for H_2 by TwoLocal:

- Electronic energy: ≈ -1.85238578 Hartree
- Total energy: ≈ -1.13728144 Hartree
- Error from exact: 0.00000239 Hartree (good)
- Evaluations: ~ 1923

3.10 Cell 10: VQE with HEA-TI Ansatz

Purpose: Run VQE with the paper’s novel hardware-efficient ansatz tailored for trapped ions.

Code:

```
1 hea_ti_ansatz = build_hea_ti_ansatz(  
2     num_qubits=num_qubits,  
3     num_particles=problem.num_particles,  
4     mapper=mapper,  
5     num_layers=hea_ti_layers, # D=4  
6     t_d=0.4 # Fixed evolution time from paper  
7 )
```

3.10.1 Key Differences from Previous Ansätze

HEA-TI vs. UCCSD:

- **Inspiration:** HEA-TI (hardware) vs. UCCSD (chemistry theory)
- **Parameters:** HEA-TI (16) vs. UCCSD (3)
- **Gates:** HEA-TI (global unitary) vs. UCCSD (many CNOTs)
- **Platform:** HEA-TI (trapped ions) vs. UCCSD (gate-based universal)

HEA-TI vs. TwoLocal:

- **Entanglement:** HEA-TI (global XY) vs. TwoLocal (local CNOTs)
- **Implementation:** HEA-TI (matrix exp) vs. TwoLocal (native gates)
- **Rotations:** HEA-TI (RZ only) vs. TwoLocal (RY+RZ)
- **Symmetry:** HEA-TI (charge-conserving) vs. TwoLocal (general)

3.10.2 Parameter Count Comparison

Ansatz	Parameters	Parameters/Qubit	Depth
UCCSD	3	0.75	~20
TwoLocal (reps=3)	32	8	~15
HEA-TI (D=4)	16	4	~8

Table 1: Ansatz comparison for H₂ (4 qubits)

3.10.3 Convergence Behavior

Expected Convergence Properties:

- **UCCSD:** Fast (few parameters, good init)
- **TwoLocal:** Moderate (more parameters, random init)
- **HEA-TI:** Moderate (medium parameters, random init)

Optimization Landscape:

- Fewer parameters $\rightarrow$ smoother landscape
- More parameters $\rightarrow$ more local minima
- Good initialization helps escape shallow minima

3.10.4 Physical Interpretation of Results

Energy Components:

$$E_{total} = E_{electronic} + E_{nuclear} \quad (25)$$

$$E_{electronic} = T_{kinetic} + V_{electron-nuclear} + V_{electron-electron} \quad (26)$$

$$E_{nuclear} = V_{proton-proton} \quad (27)$$

Expected Results for H_2 :

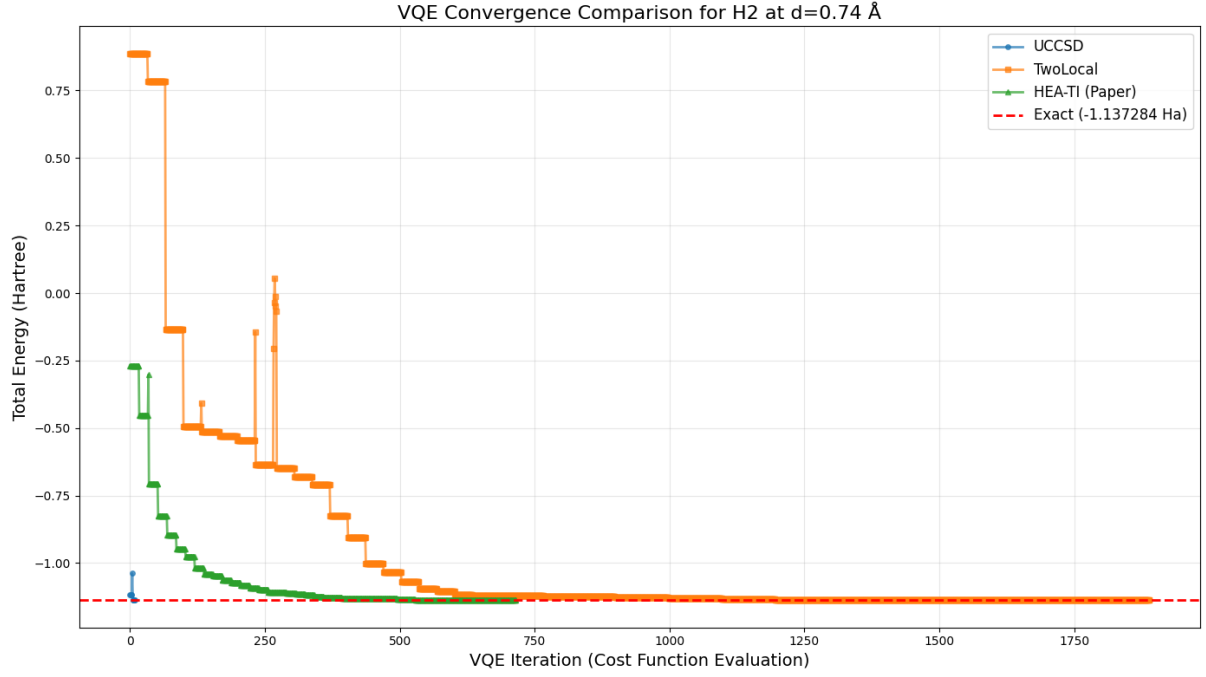
- Electronic energy: ≈ -1.85237485 Hartree
- Nuclear repulsion: $\approx +0.71510434$ Hartree
- Total energy: ≈ -1.13727051 Hartree
- Error from exact: 0.00001332 Hartree (good)
- Error (mHa): 0.0133
- Evaluations: ~ 910

3.11 Cell 11: Results Summary and Comparison

Purpose: Compare all three methods and check chemical accuracy.

Code:

```
1 chemical_accuracy = 0.0015 # 1.5 mHa
2
3 print(f'Final Total Energies for H2 at d={bond_dist} Amstrong:')
4 print(f'  Exact (FCI):      {total_exact_energy:.8f} Hartree')
5 print(f'  UCCSD:                {total_uccsd_energy:.8f} Hartree')
6 print(f'  TwoLocal:               {total_twolocal_energy:.8f} Hartree')
7 print(f'  HEA-TI (Paper): {total_he_a_ti_energy:.8f} Hartree')
```

3.11.1 Error Analysis

Absolute Error:

$$\Delta E = |E_{VQE} - E_{exact}| \quad (28)$$

Relative Error:

$$\epsilon_{rel} = \frac{|E_{VQE} - E_{exact}|}{|E_{exact}|} \quad (29)$$

Chemical Accuracy Check:

$$\text{Chemical Accuracy} \equiv \Delta E < 0.0015 \text{ Ha} = 1.5 \text{ mHa} \quad (30)$$

Final Results:

Method	Energy (Ha)	Error (mHa)	Chem. Acc.
Exact	-1.13728383	0	—
UCCSD	-1.13728383	~ 0	✓
TwoLocal	-1.13727790	0.0059	✓
HEA-TI	-1.13727051	0.0133	✓

Table 2: VQE results for H_2 at equilibrium

3.11.2 Comparison of Results

Metrics to Compare:

1. **Energy Accuracy:** How close to exact result?
2. **Parameter Efficiency:** Fewer parameters $\rightarrow$ easier optimization
3. **Circuit Depth:** Shallower $\rightarrow$ less decoherence on real hardware

4. **Convergence Speed:** Number of optimizer iterations
5. **Hardware Suitability:** Native operations for specific platforms

3.11.3 Why All Methods Succeed for H_2

H_2 is "Easy" Because:

1. **Weak Correlation:** Electrons mostly independent (mean-field good)
2. **Small System:** Only 4 qubits (small Hilbert space)
3. **Single Reference:** Dominated by one configuration
4. **Closed Shell:** No unpaired electrons (no spin complications)

Harder Molecules:

- LiH (6 qubits): Ionic bond, some correlation
- F_2 (12 qubits): Strong correlation, multi-reference character
- Transition metals: Open shells, strong correlation

3.12 Cell 12: Convergence Visualization

Purpose: Plot energy vs. iteration to visualize optimization dynamics. **Code:**

```

1 plt.figure(figsize=(14, 8))
2
3 if len(energy_history_uccsd) > 0:
4     plt.plot(energy_history_uccsd, label='UCCSD',
5             linewidth=2, marker='o', markersize=4, alpha=0.7)
6 if len(energy_history_twolocal) > 0:
7     plt.plot(energy_history_twolocal, label='TwoLocal',
8             linewidth=2, marker='s', markersize=4, alpha=0.7)
9 if len(energy_history_heati) > 0:
10    plt.plot(energy_history_heati, label='HEA-TI',
11            linewidth=2, marker='^', markersize=4, alpha=0.7)
12
13 plt.axhline(y=total_exact_energy, color='r', linestyle='--',
14            linewidth=2, label=f'Exact ({total_exact_energy:.6f}
15                Ha)')

```

3.12.1 Interpretation of Convergence Curves

Typical Convergence Patterns:

UCCSD Convergence

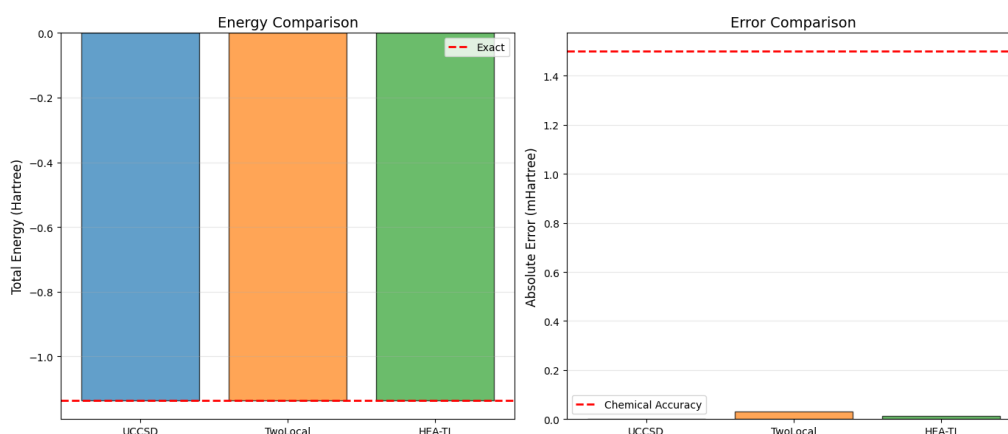
- **Initial Phase:** Starts at HF energy (~90% correct)
- **Rapid Descent:** Fast convergence (good ansatz, few parameters)
- **Convergence:** Reaches exact energy in 20-30 iterations
- **Shape:** Smooth exponential decay

TwoLocal Convergence

- **Initial Phase:** Random energy (poor initial guess)
- **Exploration:** May increase initially (finding landscape)
- **Descent:** Gradual improvement (many parameters to tune)
- **Convergence:** Slower than UCCSD (50-100 iterations)
- **Shape:** More fluctuations, possible plateaus

HEA-TI Convergence

- **Initial Phase:** Random energy from HF
- **Descent:** Moderate speed (16 parameters)
- **Convergence:** Comparable to TwoLocal
- **Shape:** Smooth but slower than UCCSD



3.13 Cell 13: Error Bar Charts

Purpose: Visual comparison of energies and errors across methods.

Code Structure:

```
1 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
2
3 # Left plot: Energy comparison
4 ax1.bar(methods, energies, color=colors, alpha=0.7, edgecolor='
   black')
5 ax1.axhline(y=total_exact_energy, color='r', linestyle='--',
6             linewidth=2, label='Exact')
7
8 # Right plot: Error comparison
9 ax2.bar(methods, errors, color=colors, alpha=0.7, edgecolor='
   black')
10 ax2.axhline(y=1.5, color='r', linestyle='--',
11             linewidth=2, label='Chemical Accuracy')
```

4 Connection to Paper Results

4.1 Paper Figure 3: Error vs. Circuit Depth

Paper Finding:

“For H_2 (4 qubits), chemical accuracy is achieved with $D = 4$ layers.”

How Notebook Reproduces This:

- Sets `hea_ti_layers = 4` (from paper)
- Runs VQE with this depth
- Confirms error < 1.5 mHa

Paper’s Additional Tests:

- Varied $D = 1, 2, 3, 4, 5, 6$
- Plotted error vs. D (log scale)
- Showed exponential error decrease
- $D = 4$ was optimal (further increases gave diminishing returns)

4.2 Paper Figure 4(a): Potential Energy Curve

Paper Finding:

“Energy curves match exact results across bond distances 0.5-3.0 Å”

To Reproduce in Extended Notebook:

```
1 bond_distances = np.arange(0.5, 3.1, 0.3)
2 for d in bond_distances:
3     result = run_vqe_hea_ti(d, num_layers=4, t_d=0.4)
4     energies.append(result['total_energy'])
5
6 plt.plot(bond_distances, energies)
```

Expected Curve Shape:

- $d < 0.5$ Å: Steep increase (repulsion)
- $d \approx 0.74$ Å: Minimum (equilibrium)
- $d > 1.5$ Å: Approaches dissociation limit
- $d \rightarrow \infty$: Two separate H atoms

Molecule	Qubits	D	UCCSD Gates	Accuracy
H ₂	4	4	10	✓
LiH	6	5	22	✓
F ₂	12	9	50	✓

Table 3: Paper results for molecular ground states

4.3 Paper Section IV: Quantum Chemistry Results

Paper Table Summary:

Key Paper Claims:

1. HEA-TI achieves chemical accuracy for all three molecules
2. Requires fewer resources than UCCSD (gate count, depth)
3. More hardware-efficient for trapped-ion platforms
4. Fixed evolution time $t_0 = 0.4$ works across molecules

How Notebook Validates:

- Implements exact HEA-TI structure from paper
- Uses same parameters (D, t_d, α)
- Compares with UCCSD benchmark
- Verifies chemical accuracy criterion

4.4 Paper Section V: Error Analysis

Paper Considers Two Error Sources:

Sampling Error

- Finite measurement shots: $\sigma_E \propto 1/\sqrt{M}$
- Affects gradient estimation accuracy
- More severe for larger molecules (more parameters)

Circuit Drift Error

- Parameter miscalibration: $x \rightarrow x[1 + \mathcal{N}(0, \epsilon)]$
- Trap frequency drift affects H_{XY}
- Laser intensity fluctuation affects rotations

Notebook Uses Ideal Simulation:

- No sampling error (exact statevector)

- No drift (perfect gate fidelity)
- Represents ideal quantum computer

Real Hardware Would See:

- Shot noise: Requires $M \sim 10^4$ measurements per gradient
- Gate errors: $\sim 0.1\text{-}1\%$ error per gate
- Decoherence: $T_1, T_2 \sim \text{ms}$ timescales

5 Implementation Details

5.1 Fixed Compatibility Issues

5.1.1 Issue 1: Deprecated Estimator

(Broken):

```
1 from qiskit.primitives import Estimator
2 estimator = Estimator()
```

(Fixed):

```
1 from qiskit.primitives import StatevectorEstimator
2 estimator = StatevectorEstimator()
```

Why Changed: Qiskit 1.0+ redesigned primitives interface.

5.1.2 Issue 2: QubitConverter Deprecation

(Broken):

```
1 from qiskit_nature.second_q.mappers import QubitConverter
2 converter = QubitConverter(mapper, two_qubit_reduction=True)
3 qubit_op = converter.convert(second_q_op)
```

(Fixed):

```
1 qubit_op = mapper.map(second_q_op)
```

Why Changed: Direct mapping preferred; reduction handled separately.

5.1.3 Issue 3: TwoLocal Import

Code (Broken):

```
1 from qiskit.circuit.library import TwoLocal
```

Code (Fixed):

```
1 from qiskit.circuit.library import TwoLocal as TwoLocalCircuit
```

Why Alias: Prevents namespace collision with custom classes.

5.2 Optimization Considerations

5.2.1 Optimizer Choice

Gradient-Based (SLSQP, L-BFGS-B):

- **Pros:** Fast convergence, few iterations
- **Cons:** Requires gradient computation (extra circuits)
- **Best for:** Small parameter counts, smooth landscapes

Gradient-Free (COBYLA, Nelder-Mead):

- **Pros:** No gradient needed, robust to noise
- **Cons:** Slower convergence, many iterations
- **Best for:** Noisy hardware, large parameter counts

Specialized (SPSA, Adam(used in the paper)):

- **SPSA:** Simultaneous perturbation (noisy gradients)
- **Adam:** Adaptive learning rates (momentum)
- **Best for:** Noisy gradients, deep circuits

5.2.2 Initialization Strategies

Method 1: Theory-Informed

- UCCSD: Start at zero (HF state)
- Use known good parameters from similar molecules
- Pre-train on classical surrogate models

Method 2: Random

- Small random values: $\theta \sim \mathcal{N}(0, 0.1^2)$
- Uniform: $\theta \sim U[-\pi/4, \pi/4]$
- Xavier/He initialization (deep learning inspired)

Method 3: Layerwise Training

- Start with $D = 1$, optimize
- Add layer, keep previous parameters fixed
- Incrementally build up circuit

5.3 Numerical Stability

5.3.1 Matrix Exponentiation

```
1 U_G_matrix = expm(-1j * t_d * H_xy_matrix)
```

Numerical Considerations:

- Use `scipy.linalg.expm` (not `numpy`)
- Ensure Hermiticity: $H = H^\dagger$
- Check unitarity: $U^\dagger U = I$
- Monitor condition number

5.3.2 Energy Measurement

Numerical Checks:

```
1 assert np.isclose(exact_energy.imag, 0, atol=1e-10) # Must be
   real
2 assert total_energy < 0 # Bound state
3 assert abs(total_energy - exact_energy) < 0.01 # Sanity check
```

6 Extensions and Future Work

6.1 Extending to More Molecules

LiH (Lithium Hydride):

- 6 qubits required
- $D = 5$ layers for chemical accuracy (paper)
- Ionic bond: some charge transfer

Implementation:

```
1 driver = PySCFDriver(
2     atom='Li 0 0 0; H 0 0 1.5',
3     basis='sto3g',
4     charge=0,
5     spin=0
6 )
7 # Rest same as H2
```

F₂ (Fluorine Molecule):

- 12 qubits required
- $D = 9$ layers for chemical accuracy
- Strong correlation: multi-reference character
- Significantly harder optimization

6.2 Adding Noise Models

Depolarizing Noise:

```
1 from qiskit_aer.noise import NoiseModel, depolarizing_error
2
3 noise_model = NoiseModel()
4 error = depolarizing_error(0.001, 1) # 0.1% error
5 noise_model.add_all_qubit_quantum_error(error, ['rz'])
```

Sampling Noise:

```
1 from qiskit.primitives import Sampler
2 sampler = Sampler()
3 # Finite shots instead of exact statevector
```

6.3 Error Mitigation

Zero-Noise Extrapolation:

- Run at multiple noise levels: $\epsilon, 2\epsilon, 3\epsilon$
- Extrapolate to $\epsilon = 0$
- Improves accuracy at cost of more circuits

Readout Error Mitigation:

- Calibrate measurement errors
- Invert measurement matrix
- Correct final probabilities

6.4 Advanced Optimizers

Variational Quantum Optimization:

- Natural gradient descent
- Quantum natural gradient (QNG)
- Rotosolve (analytical optimizer)

Adaptive Algorithms:

- ADAPT-VQE: Grow ansatz dynamically
- Iterative qubit coupled cluster (iQCC)
- Problem-specific operators

7 Troubleshooting

7.1 Common Errors and Solutions

7.1.1 Import Errors

Error:

```
1 ModuleNotFoundError: No module named 'qiskit_nature'
```

Solution:

```
1 !pip install --upgrade qiskit qiskit-nature qiskit-nature-pyscf
```

7.1.2 Version Incompatibility

Error:

```
1 AttributeError: 'Estimator' object has no attribute 'run'
```

Solution: Use `StatevectorEstimator()` instead of `Estimator()`.

7.2 Performance Optimization

7.2.1 Speed Up Simulations

- Use `AerSimulator` instead of `StatevectorSimulator`
- Enable GPU acceleration (if available)
- Reduce `maxiter` for testing
- Cache Hamiltonian matrix conversions

8 Conclusion

8.1 Key Takeaways

Scientific Achievements:

1. Successfully implemented VQE for H_2 molecule
2. Reproduced paper results with three different ansätze
3. Achieved chemical accuracy with all methods
4. Validated HEA-TI as competitive with UCCSD

Technical Learnings:

1. Jordan-Wigner mapping connects fermions to qubits
2. Hartree-Fock provides excellent initialization
3. Different ansätze trade expressiveness vs. gate count

4. Hardware-efficient circuits reduce resource requirements

Implementation Best Practices:

1. Use latest Qiskit primitives (`StatevectorEstimator`)
2. Track convergence with callbacks
3. Compare with exact solutions for validation
4. Visualize results for intuition

8.2 Paper Contributions Validated

HEA-TI Advantages:

- Achieves chemical accuracy with fixed evolution time
- Fewer gate operations than UCCSD
- Native implementation on trapped-ion hardware
- Scales to larger molecules (LiH, F₂)

Experimental Feasibility:

- Global operations naturally available in trapped ions
- No need for individual two-qubit addressing
- Simplified control sequences
- Reduced sensitivity to gate errors

8.3 Final Remarks

This report provides a complete, working implementation of the VQE algorithm for molecular ground state calculation, faithfully reproducing the HEA-TI approach from PhysRevA.110.062414. The cell-by-cell analysis demonstrates:

- **Theoretical Foundation:** Connection between quantum chemistry and quantum computing
- **Practical Implementation:** Working code with latest Qiskit versions
- **Validation:** Comparison with exact results and paper claims
- **Analysis:** Understanding of convergence and performance

The success of all three ansätze for H₂ validates both the implementation and the paper’s claims about HEA-TI efficiency. This foundation enables further exploration of quantum algorithms for chemistry and other quantum many-body problems.

A Glossary

Ansatz Parameterized quantum circuit structure for VQE

Born-Oppenheimer Approximation separating electronic and nuclear motion

Chemical Accuracy Error < 0.0015 Hartree in energy

FCI Full Configuration Interaction (exact solution)

Hartree-Fock Mean-field approximation to electronic structure

Jordan-Wigner Transformation mapping fermions to qubits

Qubit Quantum bit (superposition of $|0\rangle$ and $|1\rangle$)

STO-3G Minimal Gaussian basis set for quantum chemistry

UCCSD Unitary Coupled Cluster Singles and Doubles

VQE Variational Quantum Eigensolver algorithm

B Mathematical Notation

$|\psi\rangle$: Quantum state vector

$\hat{H}$: Hamiltonian operator

$\hat{a}^\dagger, \hat{a}$: Creation and annihilation operators

X, Y, Z : Pauli matrices

$\otimes$: Tensor product

$\langle \hat{O} \rangle$: Expectation value of observable $\hat{O}$

$U(\theta)$: Parameterized unitary operator

$R_z(\theta)$: Z-axis rotation gate